UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



Sistemas Operativos

MiniSO

Autores:

- PACCI PÉREZ, Giorgini Disalvatore 2024-119035
  gpaccip@unjbg.edu.pe
- TËLLEZ MESA Alexander Jesús 2024-119038
  atellezm@unjbg.edu.pe
- MENESES CHAUPI, Grecia Irene 2021-119055
  gimenesesc@unjbg.edu.pe

TACNA - PERÚ

1. Introducción

El presente informe detalla el diseño, implementación y resultados de MiniSO, una aplicación modular desarrollada en C++ que simula la arquitectura fundamental de un sistema operativo mediante un entorno de consola en Linux. A lo largo de este documento, se explorará la integración práctica de los principales conceptos de la asignatura de Sistemas Operativos, dividiendo el proyecto en tres componentes esenciales: la gestión y orquestación de procesos reales utilizando llamadas nativas del sistema, la sincronización concurrente de hilos para resolver el problema clásico del productor-consumidor mitigando condiciones de carrera, y la simulación de técnicas de administración de memoria mediante particionamiento dinámico (First-Fit) y paginación. El objetivo central del proyecto es demostrar, a través de escenarios de prueba configurables, una comprensión profunda del funcionamiento interno del sistema operativo, evidenciando la capacidad del equipo para resolver problemas complejos de programación de sistemas y aplicar mecanismos de control sobre los recursos de hardware compartidos.

2. Diseño de la solución

2.1. Arquitectura general del MiniSO

MiniSO presenta una arquitectura modular desarrollada en C++, organizada en tres módulos: gestión de procesos, hilos y concurrencia, y gestión de memoria. Estos módulos se integran mediante un menú principal que permite al usuario acceder a sus funcionalidades desde la consola de Linux.

El diagrama de arquitectura general del proyecto se presenta en el *Anexo 1, Figura 1.*

2.2. Diseño del módulo de gestión de procesos

El módulo de gestión de procesos se diseñó como una unidad independiente del resto del programa, compuesta por un archivo de cabecera (procesos.hpp) y uno de implementación (procesos.cpp). La cabecera expone únicamente la función menuProcesos(), que actúa como punto de entrada, mientras que las demostraciones quedan encapsuladas en el archivo de implementación. Desde este menú el usuario puede elegir:

- Opción 1, Proceso único con wait(): el padre crea un hijo con fork(), este ejecuta el comando echo mediante execvp() y el padre espera con wait() a que termine para informar su PID y su código de retorno.
- Opción 2, Procesos concurrentes con waitpid(): el padre crea dos hijos que se ejecutan de forma concurrente (uno con ls -l y otro con uptime) y espera a cada uno por separado con waitpid() para mostrar su estado de salida.

La opción 0 permite volver al menú principal. Al terminar cada demostración el control regresa al menú, que se repite hasta que el usuario decide salir.

En ambos casos se siguió el esquema clásico de Linux: fork() crea al hijo, execvp() reemplaza su imagen por otro programa y el padre espera su finalización. Cada mensaje en pantalla se identifica con un prefijo ([PADRE],

[HIJO 1], [HIJO 2]) junto con su PID y PPID, lo que hace visible la relación jerárquica entre procesos. Además, el diseño contempla el manejo de errores en fork() y execvp() mediante perror(), y la validación de la entrada del menú para evitar ciclos infinitos ante datos inválidos.

2.3. Diseño del módulo de hilos y concurrencia

El módulo de Gestión de Hilos y Concurrencia tiene como finalidad implementar y demostrar la ejecución de múltiples hilos dentro de un mismo proceso, permitiendo que diferentes tareas se ejecuten de manera concurrente y compartan recursos de forma controlada. Para ello, se implementa el problema clásico de productor-consumidor mediante un búfer compartido de capacidad limitada, donde los productores se encargan de generar elementos y almacenarlos, mientras que los consumidores retiran dichos elementos para procesarlos.
El diseño del módulo se divide en tres componentes:

- Productores: Son los hilos encargados de generar elementos e introducirlos en el búfer compartido. Cada productor tiene asignada una cantidad de elementos y trabaja de manera independiente. Si el búfer alcanza su capacidad máxima, el productor debe esperar hasta que exista espacio disponible.
- Consumidores: Son los hilos encargados de retirar los elementos almacenados en el búfer. Estos trabajan concurrentemente con los productores y, cuando el búfer se encuentra vacío, esperan hasta que se produzcan nuevos elementos. Cuando todos los productores terminan y ya no quedan elementos por consumir, los consumidores finalizan su ejecución.
- Búfer compartido: Es la estructura central del módulo y permite almacenar temporalmente los elementos producidos. Se implementa mediante una cola (queue), que trabaja bajo el principio FIFO (First In, First Out), es decir, el primer elemento que ingresa es el primero en salir. Su capacidad es configurable y su acceso está protegido mediante mecanismos de sincronización.

2.4. Diseño del módulo de gestión de memoria

El módulo de gestión de memoria permite simular la asignación y liberación de espacios de memoria, así como la traducción de direcciones lógicas a físicas. El módulo se divide en dos partes:

- **Particionamiento dinámico (First-Fit):** Permite configurar la memoria, asignar y liberar procesos, identificar la fragmentación externa y unir bloques libres. Según Sáiz de la Peña y Sáiz Díez (2025), en la asignación continua un proceso necesita un espacio libre suficientemente grande, aunque existan otros espacios disponibles que juntos superen el tamaño solicitado.
- **Paginación:** Permite configurar páginas y marcos físicos, realizar asignaciones y traducir direcciones. Sáiz de la Peña y Sáiz Díez (2025) explican que la memoria paginada utiliza marcos de tamaño

fijo para almacenar las páginas de los procesos, sin necesidad de que se encuentren en posiciones contiguas.

(Ver Anexo 4, Figura 4.1).

3. Implementación

3.1. Implementación de gestión de procesos

3.1.1. Descripción general

Para esta práctica se desarrolló en C++ el módulo procesos (archivos procesos.hpp y procesos.cpp), que se integra a un menú principal mediante la función menuProcesos(). El módulo demuestra la creación, ejecución y sincronización de procesos en Linux usando llamadas al sistema POSIX.
Un proceso es una instancia de un programa en ejecución, y cada uno se identifica con un PID. Todo proceso tiene además un PPID, que es el PID del proceso que lo creó. Así se forma la jerarquía padre-hijo descrita en la teoría.
El módulo ofrece dos demostraciones desde un menú interactivo:

Opción	Función	Llamada de sincronización
1	ejecucionConWaitGenerico()	wait()
2	ejecucionConWaitpidEspecifico()	waitpid()
0	Volver al menú principal	—

3.1.2. Funciones POSIX utilizadas

Función	Uso en la implementación
getpid()	Muestra el PID del proceso padre y de los hijos.
getppid()	Muestra, desde el hijo, el PID de su padre.
fork()	Crea un nuevo proceso hijo, copia del padre.
execvp()	Reemplaza la imagen del hijo por otro programa (echo, ls, uptime).
wait()	El padre espera a que termine cualquier hijo.
waitpid()	El padre espera a que termine un hijo específico, indicado por su PID.

3.1.3. Interfaz del módulo (procesos.hpp)

El archivo de cabecera usa guardas de inclusión (#ifndef / #define / #endif) y declara solo void menuProcesos();. Esa es la única función pública. Las dos demostraciones quedan encapsuladas en procesos.cpp, y el menú principal no necesita conocer los detalles internos.

3.1.4. Demostración 1: proceso único con wait()

3.1.4.1. Funcionamiento

1. El padre imprime su PID con getpid().
2. Llama a fork(), que devuelve un valor distinto según quién lo reciba:
   – Menor que 0: hubo un error y se informa con perror().
   – Igual a 0: es el proceso hijo.
   – Mayor que 0: es el proceso padre, y el valor devuelto es el PID del hijo.
3. El hijo imprime su PID y su PPID, y luego ejecuta echo con execvp(). Con esta llamada el hijo reemplaza su imagen por la del comando echo.
4. El padre llama a wait(&status), que lo bloquea hasta que el hijo termina y devuelve el PID del hijo finalizado.
5. El padre revisa el estado de salida con las macros WIFEXITED(status) y WEXITSTATUS(status).

3.1.4.2. Fragmento clave

```
pid_t pid = fork();

if (pid == 0) {                 // HIJO
        char* args[] = {arg0, arg1, NULL};
        execvp(args[0], args);              // reemplaza la imagen del proceso
        perror("Error en execvp()");
        exit(EXIT_FAILURE);
} else {                        // PADRE
        int status;
        pid_t hijo_finalizado = wait(&status);
        if (WIFEXITED(status))
        cout << WEXITSTATUS(status);  // código de retorno del hijo
}
```

3.1.4.3. Puntos importantes

- Si execvp() tiene éxito, el código posterior dentro del hijo nunca se ejecuta, porque la imagen del proceso fue reemplazada. Por eso el perror() y el exit(EXIT_FAILURE) solo se alcanzan si hubo un error.
- El PPID impreso por el hijo coincide con el PID impreso por el padre, lo que confirma la relación jerárquica.
- Se espera un código de retorno 0, lo que indica que echo terminó correctamente.

3.1.5. Demostración 2: procesos concurrentes con waitpid()

3.1.5.1. Funcionamiento

El padre crea dos hijos con dos llamadas consecutivas a fork():

Proceso	Comando	Función
Hijo 1	ls -l	Lista el contenido del directorio con detalle.
Hijo 2	uptime	Muestra el tiempo de actividad del sistema.

Los hijos se ejecutan de forma concurrente. El sistema operativo reparte el procesador entre ellos y el padre mediante fracciones de tiempo (time slices), de modo que parece que avanzan simultáneamente.
El padre espera a cada hijo por separado con waitpid(pid1, ...) y waitpid(pid2, ...), y así controla cuál termina y con qué estado. Al final muestra el estado de salida de cada hijo.

3.1.5.2. Fragmento clave

```
int status1, status2;
waitpid(pid1, &status1, 0);   // espera específicamente al Hijo 1
waitpid(pid2, &status2, 0);   // espera específicamente al Hijo 2
```

3.1.5.3. Puntos importantes

- Los argumentos de cada comando se preparan antes de los fork(), en los arreglos args1 y args2, que terminan en NULL como exige execvp().
- Como los hijos corren concurrentemente, el orden en que aparecen sus mensajes en pantalla puede variar entre ejecuciones. Eso depende del planificador del sistema operativo.
- La sincronización del padre sí es determinista: no imprime el resumen final hasta que ambos hijos terminaron.

3.1.6. Comparación entre wait() y waitpid()

Característica	wait()	waitpid()
Hijo que espera	**Cualquiera que termine**	**Uno específico (por PID)**
Control sobre el orden	**No**	**Sí**
Caso de uso en el código	**Un solo hijo**	**Varios hijos concurrentes**
Retorno	**PID del hijo finalizado**	**PID del hijo esperado**

3.1.7. Menú de control

menuProcesos() usa un ciclo do...while que repite el menú hasta que el usuario elige 0. También valida la entrada: si el usuario escribe algo que no es un número, limpia el error de cin con cin.clear(), descarta el resto de la línea con getline() y vuelve a mostrar el menú. Esto evita que el programa quede en un ciclo infinito por una entrada inválida.

3.2. Implementación de hilos y concurrencia

El módulo fue desarrollado en lenguaje C++, utilizando la biblioteca POSIX Threads mediante la inclusión de pthread.h. Esta biblioteca permite crear, ejecutar y sincronizar múltiples hilos dentro de un mismo proceso.
Para su implementación se utilizaron diferentes bibliotecas y estructuras de datos:

- pthread.h: Proporciona las funciones necesarias para crear y administrar hilos, utilizar mutex y trabajar con variables de condición.
- queue: Permite implementar el búfer compartido mediante una cola de tipo FIFO.
- vector: Permite almacenar los identificadores de los hilos productores y consumidores.
- unistd.h: Proporciona la función usleep(), utilizada para introducir pequeños intervalos de tiempo y simular los tiempos de producción y consumo.
- iostream: Permite mostrar los mensajes de configuración, ejecución y resultados en consola.

3.2.1. Estructuras y mecanismos de sincronización

Para controlar la concurrencia se implementaron los siguientes elementos:

Elemento	**Función dentro del módulo**
queue<int> buffer	Almacena los elementos producidos que esperan ser consumidos.
pthread_mutex_t mtx	Controla el acceso exclusivo al búfer y sus contadores.
pthread_cond_t cv_espacio_disponible	Permite que los productores esperen cuando el búfer está lleno.
pthread_cond_t cv_elemento_disponible	Permite que los consumidores esperen cuando el búfer está vacío.
pthread_t	Representa los hilos productores y consumidores.
pthread_create()	Crea y ejecuta cada hilo.
pthread_join()	Espera la finalización de los hilos.

Las variables de condición trabajan conjuntamente con el mutex. Cuando un hilo ejecuta pthread_cond_wait(), libera temporalmente el mutex y queda bloqueado hasta recibir una señal. Posteriormente, vuelve a adquirir el mutex y comprueba nuevamente la condición antes de continuar.

3.2.2. Funciones implementadas

3.2.2.1 Clase BufferCompartido

Se encarga de administrar el búfer, controlar su capacidad y coordinar las operaciones de producción y consumo mediante los mecanismos de sincronización.

Función	Descripción
BufferCompartido()	Inicializa la capacidad del búfer, los contadores y la cantidad de productores activos. También inicializa el mutex y las variables de condición.
~BufferCompartido()	Libera los recursos de sincronización mediante la destrucción de las variables de condición y el mutex.
producir()	Introduce un elemento en el búfer. Si está lleno, bloquea al productor hasta que exista espacio disponible.
consumir()	Retira un elemento del búfer. Si está vacío, espera hasta recibir un nuevo elemento o hasta que finalicen todos los productores.
finalizarProductor()	Registra la finalización de un productor y notifica a los consumidores cuando ya no quedan productores activos.
getProducidos()	Devuelve la cantidad total de elementos producidos.
getConsumidos()	Devuelve la cantidad total de elementos consumidos.

3.2.2.2 Funciones de ejecución de los hilos

Función	Descripción
ejecutarProductor()	Ejecuta el trabajo asignado a cada productor, genera sus elementos, los introduce en el búfer y registra su finalización.
ejecutarConsumidor()	Ejecuta el trabajo de cada consumidor, retirando elementos del búfer hasta que no

	queden más elementos disponibles ni productores activos.

3.2.2.3 Funciones principales del módulo

Función	Descripción
ejecutarProductorConsumidor()	Solicita los datos de configuración, valida los parámetros, crea los hilos, espera su finalización y presenta el resumen de resultados.
menuHilosYConcurrencia()	Muestra el menú del módulo y permite al usuario ingresar al escenario productor-consumidor o regresar al menú principal.

3.3. Implementación de gestión de memoria

Este módulo se desarrolló en C++, mediante funciones y estructuras de datos que permiten simular el particionamiento dinámico y la paginación. Todo el código se encuentra en el archivo memoria.cpp.

3.3.1. Particionamiento dinámico (First-Fit)

Para implementar el particionamiento dinámico se utilizó una estructura llamada Bloque, que almacena la dirección inicial, el tamaño, el estado y el nombre del proceso. Estos datos se guardan en un vector que organiza los bloques libres y ocupados.

La función asignarProceso() utiliza el algoritmo First-Fit para buscar el primer bloque libre con espacio suficiente. Si queda memoria disponible después de la asignación, el bloque se divide.

También se implementaron las funciones liberarProceso() y unirBloques(), que permiten liberar y reutilizar espacios de memoria, uniendo los bloques libres contiguos.

Además, se implementó la detección de fragmentación externa. Según Sáiz de la Peña y Sáiz Díez (2025), en la memoria continua un proceso necesita un único espacio libre suficientemente grande, aunque la suma de los espacios disponibles sea suficiente. Por ello, el programa muestra un mensaje cuando no puede realizar la asignación debido a esta situación.

3.3.2. Paginación

Para implementar la paginación se utilizó un vector de números enteros como tabla de páginas. Cada posición guarda el marco físico asignado y el valor -1 indica que una página no tiene un marco.

Según Sáiz de la Peña y Sáiz Díez (2025), la memoria paginada utiliza marcos de tamaño fijo para almacenar las páginas de los procesos. Teniendo en cuenta ello, el programa permite asignar páginas de forma automática o manual y mostrar su distribución.

La función traducirDireccion() calcula el número de página y su desplazamiento, consulta la tabla y obtiene la dirección física. También se implementaron validaciones para direcciones inválidas, páginas no asignadas y el cálculo de la fragmentación interna.

3.3.3. Funciones implementadas

- **asignarProceso():** Asigna memoria utilizando el algoritmo First-Fit.
- **liberarProceso():** Libera la memoria asignada a un proceso.
- **unirBloques():** Une los bloques libres contiguos.
- **mostrarMemoria():** Muestra los bloques libres y ocupados.
- **asignarPaginasAutomaticamente():** Asigna las páginas a los primeros marcos disponibles.
- **asignarPaginasManualmente():** Permite seleccionar el marco físico de cada página.
- **mostrarPaginacion():** Muestra las tablas de páginas y marcos físicos.
- **traducirDireccion():** Calcula la dirección física a partir de una dirección lógica.

4. Pruebas y resultados

4.1. Pruebas y resultados de gestión de procesos

Caso de prueba	Datos de entrada	Resultado esperado	Resultado obtenido
Proceso único con wait()	Opción 1 del menú.	El padre muestra su PID y crea un hijo. El hijo muestra su PID y su PPID (igual al PID del padre) y ejecuta `echo` con `execvp()`. El padre espera con `wait()` e informa el PID del hijo finalizado y su código de retorno 0.	(*Ver anexo 2, figura 2.1.)*
Procesos concurrentes con waitpid()	Opción 2 del menú.	El padre crea dos hijos concurrentes: el Hijo 1 ejecuta `ls -l` y el Hijo 2 ejecuta `uptime`. El orden de sus mensajes puede variar. El padre espera a cada uno con `waitpid()` y muestra el estado de	(*Ver anexo 2, figura 2.2.)*

		ambos con código 0.	
Salida del módulo	Opción 0 del menú.	El módulo termina y el control regresa al menú principal.	(*Ver anexo 2, figura 2.4.)*

4.2. Pruebas y resultados de gestión de hilos y concurrencia

Para comprobar el funcionamiento del módulo se plantearon diferentes casos de prueba, modificando la capacidad del búfer, la cantidad de elementos a producir y el número de productores y consumidores.

Caso de prueba	**Datos de entrada**	**Resultado esperado**	**Resultado obtenido**
Búfer reducido	Capacidad: 2. Elementos: 30. Productores: 3. Consumidores: 2.	Los productores deben esperar cuando el búfer alcanza su capacidad máxima. Los consumidores liberan espacio para continuar la producción.	(*Ver anexo 3, figura 3.1.)*
Mayor cantidad de elementos	Capacidad: 5. Elementos: 100. Productores: 4. Consumidores: 4.	Los cuatro productores y los cuatro consumidores trabajan concurrentemente hasta completar la producción y el consumo de los 100 elementos.	(*Ver anexo 3, figura 3.2.)*
Mayor cantidad de productores	Capacidad: 3. Elementos: 50. Productores: 5. Consumidores: 2.	Se distribuyen los elementos entre los productores y los consumidores retiran los elementos disponibles, respetando la capacidad del búfer.	(*Ver anexo 3, figura 3.3.)*

4.3. Pruebas y resultados de gestión de memoria

4.3.1. Pruebas de particionamiento dinámico (First-Fit)

Caso de prueba	Datos de entrada	Resultado esperado	Resultado obtenido
Asignación de procesos	P1=50, P2=70, P3=60, P4=40	Asignar los cuatro procesos y dejar 80 MiB libres.	Se asignaron los cuatro procesos y quedaron 80 MiB libres. (*Ver anexo 4, figura 4.2.)*
Liberación de memoria	Liberar P2	Dos bloques libres de 70 y 80 MiB	Se obtuvieron dos bloques libres separados. (*Ver Anexo 4, figura 4.3.)*
Fragmentación externa	Solicitar P5=100 MiB	La asignación no se logra.	Se encontró fragmentación externa. *(Ver Anexo 4, figura 4.4.)*
Unión de bloques	Liberar P3	Los espacios continuos se unen.	Se formó un bloque libre de 130 MiB. (*Ver Anexo 4, figura 4.5.)*
Reutilización de memoria	Asignar P5=100 MiB	Se asigna P5 y deja 30 MiB libres.	P5 fue asignado correctamente. (*Ver Anexo 4, figura 4.6.)*
Reinicio	Reiniciar memoria	Se libera los 300 MiB de la memoria.	Toda la memoria queda libre y se eliminan las asignaciones. (*Ver Anexo 4, figura 4.7.)*

4.3.2. Pruebas de paginación

Caso de prueba	Datos de entrada	Resultado esperado	Resultado obtenido
Asignación manual	P1=50, P2=70, P3=60, P4=40	Se espera asignar las 3 páginas a sus respectivos marcos.	Se asignaron las 3 páginas correctamente. *(Ver Anexo 4, Figura 4.8).*
Traducción	Dirección lógica 130	Se espera obtener 194	La dirección física fue 194. (*Ver Anexo 4, figura 4.9.)*

5. Dificultades encontradas

- Durante el desarrollo de la práctica, la principal dificultad residió en la adaptación al entorno POSIX en Linux (Ubuntu) para el uso de librerías como <unistd.h> y <sys/wait.h>, ausentes en Windows nativo. Asimismo, representó un reto ajustar el manejo de arreglos de caracteres en execvp() para evitar advertencias de compilación con g++, comprender la sustitución completa del espacio de memoria del proceso hijo al invocar dicha función y

gestionar adecuadamente la sincronización con wait() y waitpid() para prevenir la creación de procesos en estado zombi.

- Durante el desarrollo del módulo de hilos y concurrencia se identificaron algunos aspectos que requirieron atención para garantizar el correcto funcionamiento de la ejecución concurrente como la necesidad de coordinar el comportamiento de los hilos cuando el búfer se encontraba lleno o vacío. Para ello se incorporaron variables de condición que permiten bloquear temporalmente los hilos y reanudarlos cuando se cumple la condición necesaria.
- Al realizar la gestión de memoria, una de las dificultades fue implementar y organizar las funciones para que cada operación funcionará correctamente. De igual forma, se presentaron dificultades al crear el archivo de cabecera memoria.hpp y adaptar el código para integrarlo con los demás módulos.

6. Conclusiones

- Se logró verificar de manera práctica el ciclo de vida y la gestión concurrente de procesos en Linux mediante la implementación de las seis funciones POSIX requeridas (getpid, getppid, fork, exec, wait y waitpid). Se comprobó que fork() permite la ejecución independiente de comandos del sistema en paralelo, mientras que la diferencia fundamental entre wait() y waitpid() radica en que la primera suspende al proceso padre hasta la finalización de cualquier hijo, mientras que la segunda permite una sincronización explícita y ordenada sobre un identificador de proceso (PID) específico.
- Para el módulo de hilos y concurrencia se implementó satisfactoriamente el problema de productor-consumidor mediante múltiples hilos utilizando la biblioteca POSIX Threads en C++. Esto permitió demostrar cómo diferentes tareas pueden ejecutarse concurrentemente dentro de un mismo proceso y compartir recursos.
- La implementación del módulo de gestión de memoria permitió comprender mejor cómo funcionan los mecanismos de particionamiento dinámico y paginación. De igual forma, mediante las pruebas, se comprobó la importancia de reutilizar los espacios libres, controlar la fragmentación y traducir correctamente las direcciones de memoria.

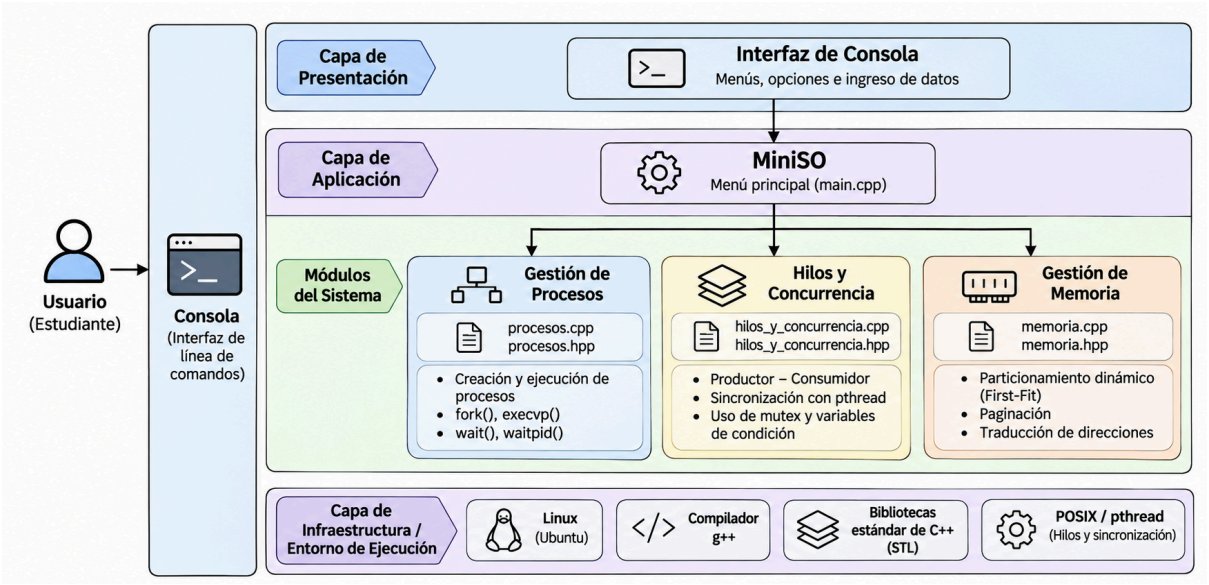
7. Referencias bibliográficas

Sáiz de la Peña, J., & Sáiz Díez, J. M. (2025). Gestión de procesos y gestión de memoria no virtual (herramientas y problemas resueltos). Universidad de Burgos. https://doi.org/10.36443/9791387585143

8. Anexos

Anexo 1. Arquitectura de software del MiniSO

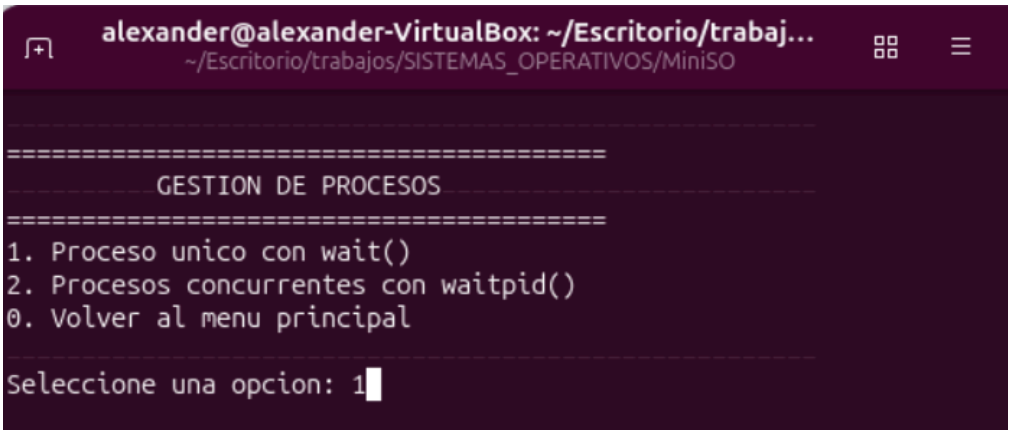
Figura 1. Arquitectura de software del proyecto MiniSO



Nota. El diagrama muestra la estructura general del sistema, el menú principal y los tres módulos implementados: gestión de procesos, hilos y concurrencia, y gestión de memoria.

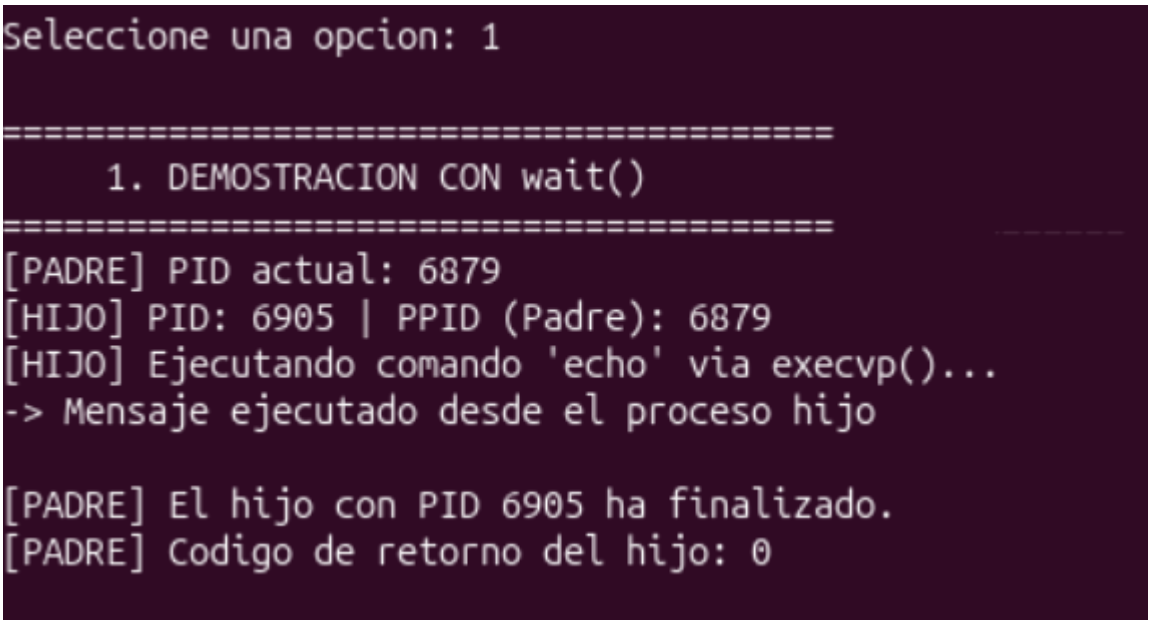
Anexo 2. Gestión de procesos

Figura 2.1 Compilación del código fuente C++ y puesta en marcha en Linux Ubuntu.



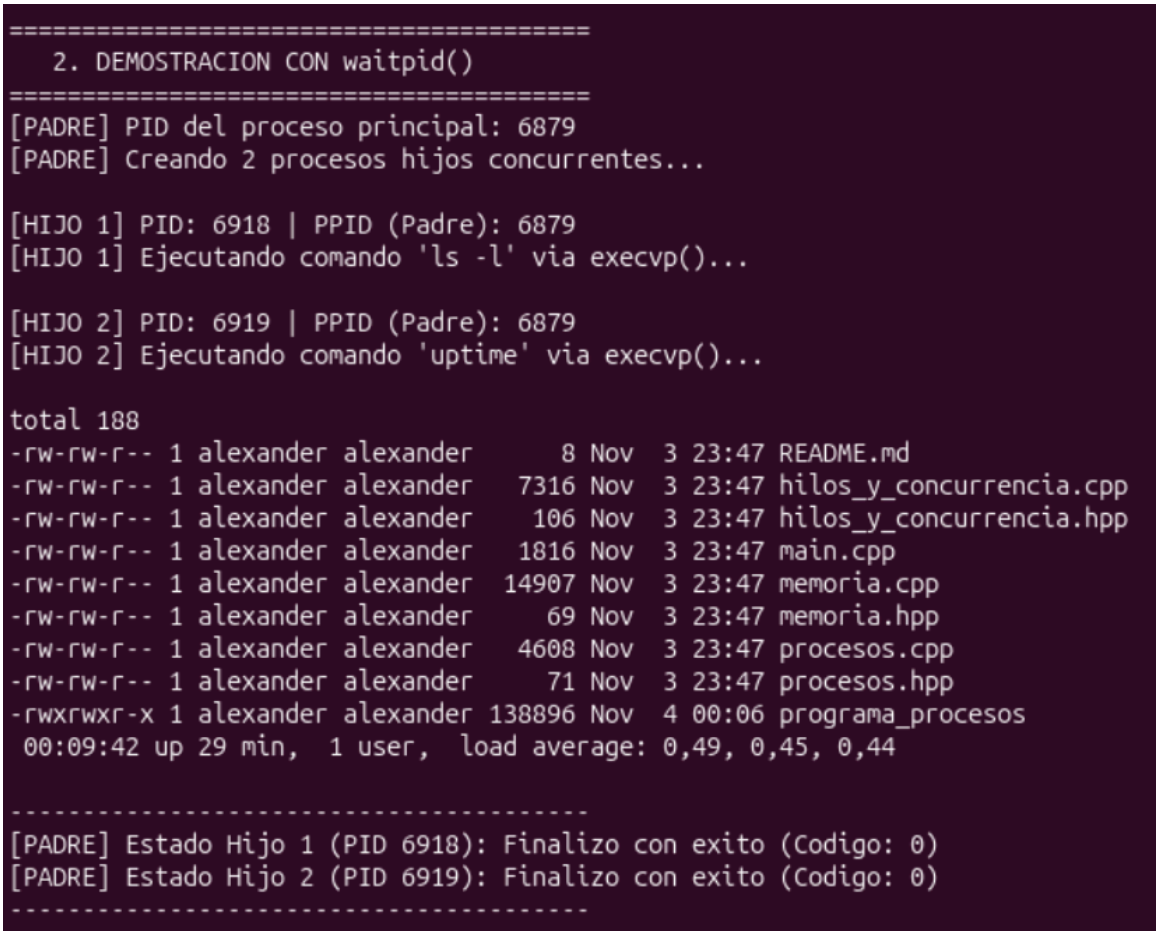
Nota. Proceso de compilación del proyecto mediante el compilador GNU (g++) en un entorno Ubuntu Linux. Muestra la ausencia de advertencias de compilación (warnings) y el despliegue del menú interactivo principal de la aplicación.

Figura 2.2 Ejecución de un proceso único mediante llamadas POSIX fork(), execvp() y wait().



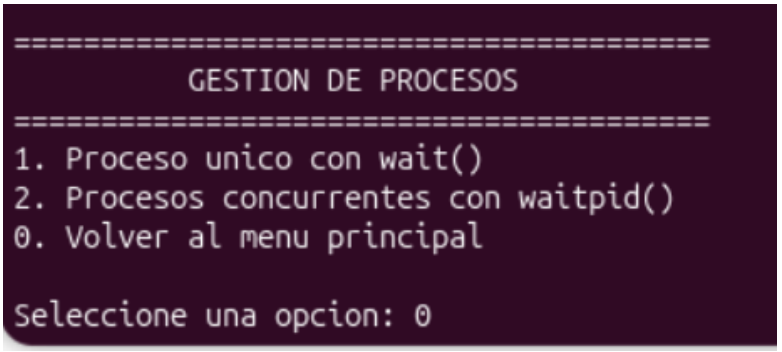
Nota. Salida producida al seleccionar la opción 1. Se observa la duplicación del proceso padre (PID 18420) para generar el proceso hijo (PID 18421). El hijo verifica su parentesco imprimiendo el PID de su padre mediante getppid() y reemplaza su segmento de memoria mediante execvp() ejecutando el comando echo. El proceso padre entra en estado de bloqueo con wait() hasta recibir la señal de terminación del hijo, capturando el código de salida 0 a través de la macro WEXITSTATUS.

Figura 2.3 Ejecución concurrente de múltiples procesos hijos y sincronización explícita con waitpid().



Nota. Salida obtenida al seleccionar la opción 2. El proceso padre bifurca dos procesos hijos concurrentes (PIDs 18422 y 18423). Cada hijo sustituye de forma independiente su espacio de direccionamiento por los binarios del sistema operativo /bin/ls y /usr/bin/uptime. El proceso padre sincroniza su finalización de manera controlada invocando waitpid() de forma dirigida para cada identificador numérico de proceso.

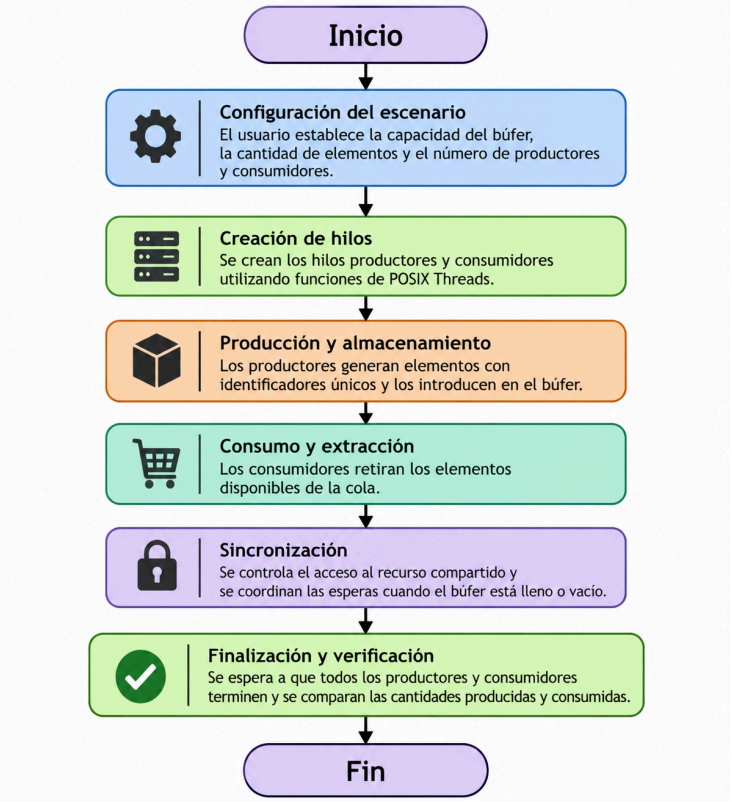
Figura 2.4 Finalización limpia y retorno de control en la consola de comandos.



Nota. Cierre controlado del ciclo de ejecución tras ingresar la opción `0`. Muestra la liberación de recursos del sistema sin la existencia de procesos bloqueados, huérfanos ni zombis.

Anexo 3. Gestión de hilos y concurrencia

Figura 3.1. Division del modulo de hilos y concurrencia



Nota. Diagrama del módulo hilos y concurrencia donde se divide en las siguientes etapas

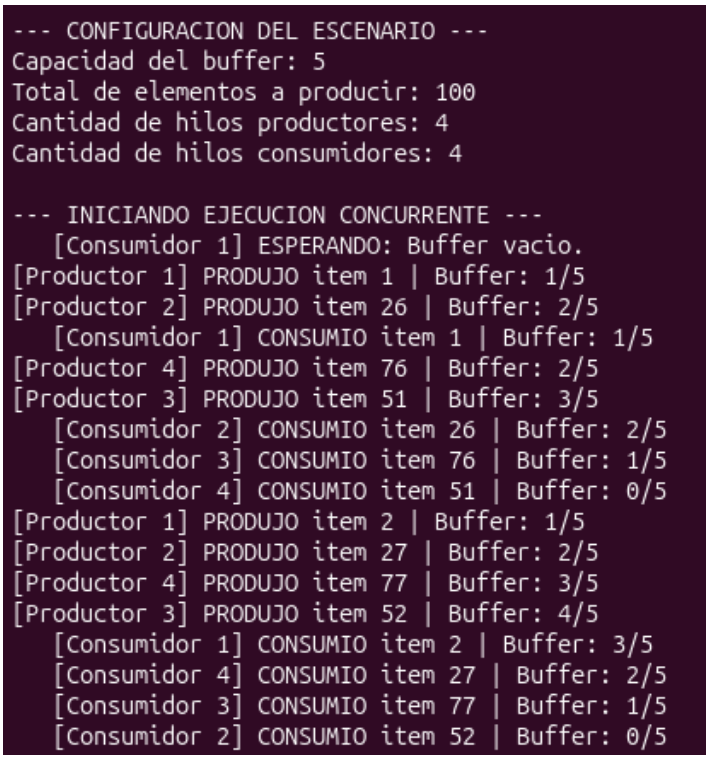
Figura 3.2. Búfer reducido

```
--- CONFIGURACION DEL ESCENARIO ---
Capacidad del buffer: 2
Total de elementos a producir: 30
Cantidad de hilos productores: 3
Cantidad de hilos consumidores: 2

--- INICIANDO EJECUCION CONCURRENTE ---
   [Consumidor 1] ESPERANDO: Buffer vacio.
   [Consumidor 2] ESPERANDO: Buffer vacio.
[Productor 1] PRODUJO item 1 | Buffer: 1/2
[Productor 2] PRODUJO item 11 | Buffer: 2/2
[Productor 3] ESPERANDO: Buffer lleno (2/2).
   [Consumidor 2] CONSUMIO item 1 | Buffer: 1/2
[Productor 3] PRODUJO item 21 | Buffer: 2/2
   [Consumidor 1] CONSUMIO item 11 | Buffer: 1/2
[Productor 3] PRODUJO item 22 | Buffer: 2/2
[Productor 2] ESPERANDO: Buffer lleno (2/2).
[Productor 1] ESPERANDO: Buffer lleno (2/2).
```

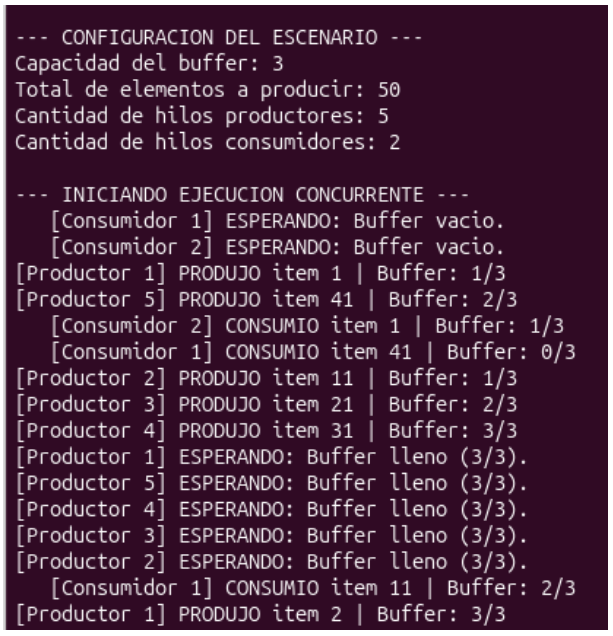
Nota. Se observa cómo los productores introducen elementos en el búfer y los consumidores los retiran de manera concurrente. Los mensajes muestran los identificadores de los elementos y la cantidad de espacios ocupados en el búfer después de cada operación.

Figura 3.3. Mayor cantidad de elementos



Nota. Esta evidencia permite comprobar el funcionamiento de las variables de condición. Cuando el búfer está lleno, los productores quedan esperando hasta que se libere espacio; cuando está vacío, los consumidores esperan hasta que se produzcan nuevos elementos.

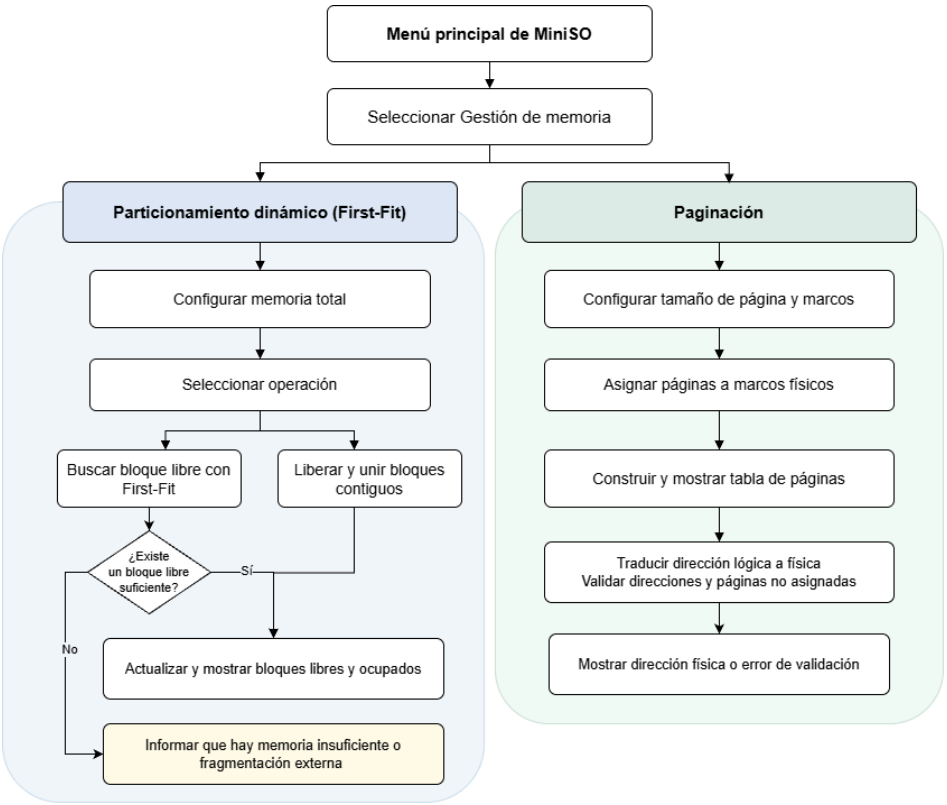
Figura 3.3. Mayor cantidad de productores



Nota. Se observa que los productores y consumidores terminan sus actividades y que el programa presenta la cantidad total de elementos producidos y consumidos. La coincidencia de ambas cantidades permite verificar que el procesamiento se completó sin pérdidas en las operaciones de la cola.

Anexo 4. Gestión de memoria

Figura 4.1. Diagrama de funcionamiento del módulo de gestión de memoria



Nota. El diagrama muestra el funcionamiento del módulo de gestión de memoria, incluyendo los procesos de asignación y liberación mediante First-Fit, así como la asignación de páginas y la traducción de direcciones lógicas a físicas

Figura 4.2. Asignación de procesos mediante First-Fit

```
Seleccione una opcion: 1
Nombre del proceso: P4
Tamano del proceso (MiB): 40
Proceso asignado correctamente.

Presione Enter para continuar...

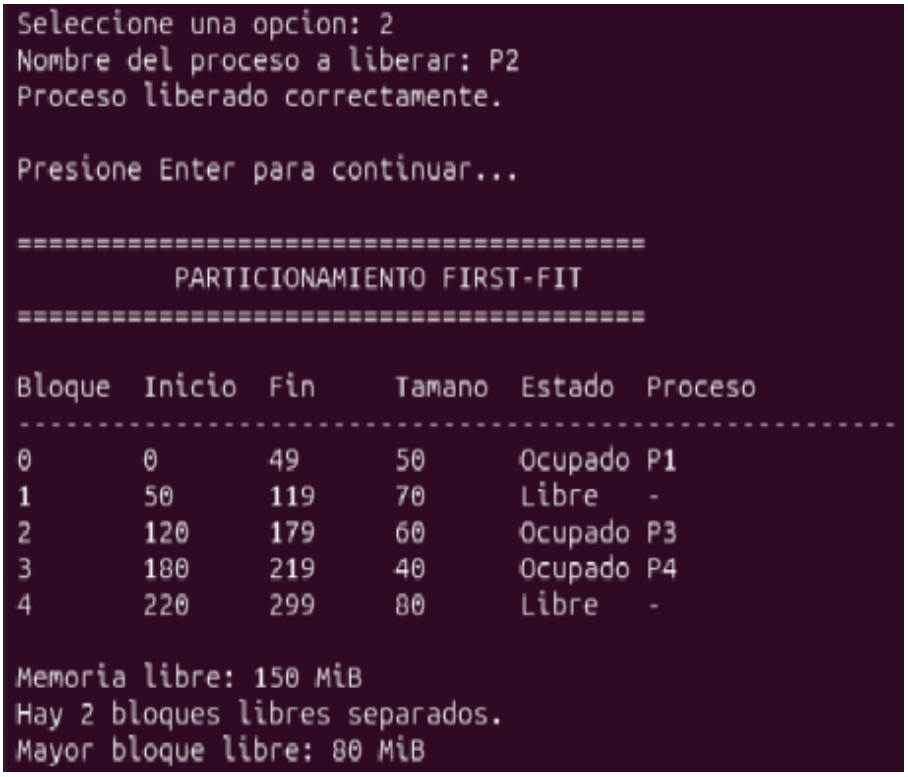
========================================
          PARTICIONAMIENTO FIRST-FIT
========================================

Bloque  Inicio  Fin     Tamano  Estado  Proceso
--------------------------------------------------------
0       0       49      50      Ocupado P1
1       50      119     70      Ocupado P2
2       120     179     60      Ocupado P3
3       180     219     40      Ocupado P4
4       220     299     80      Libre   -

Memoria libre: 80 MiB
```

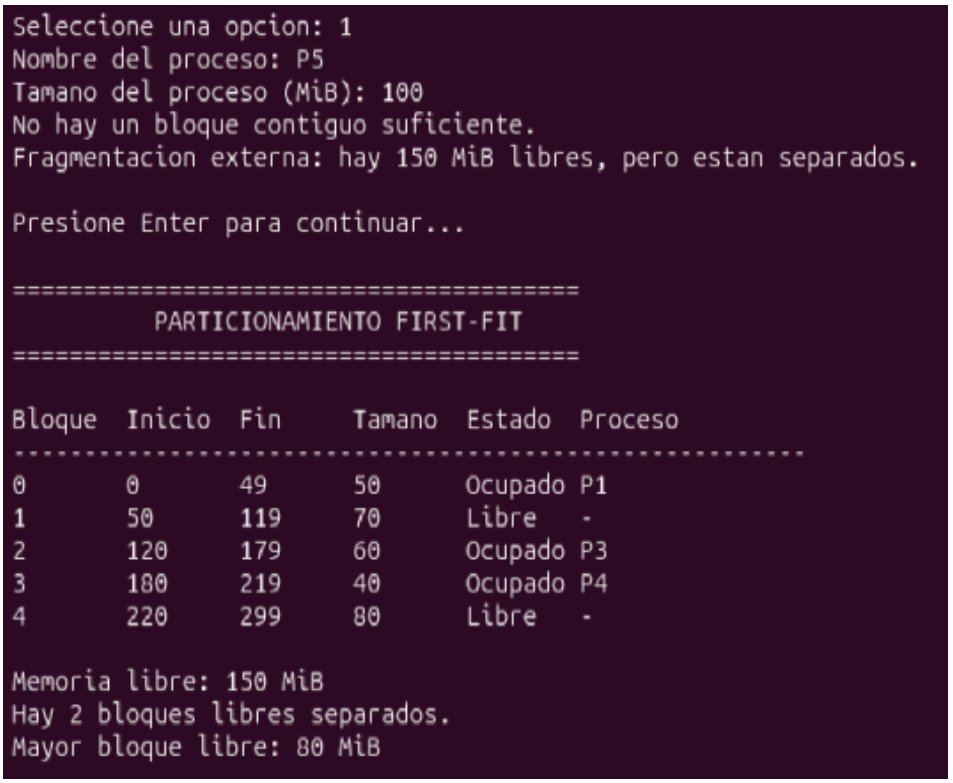
Nota. Se asignaron P1, P2, P3 y P4 en una memoria de 300 MiB, quedaron 80 MiB libres.

Figura 4.3. Liberación del proceso P2



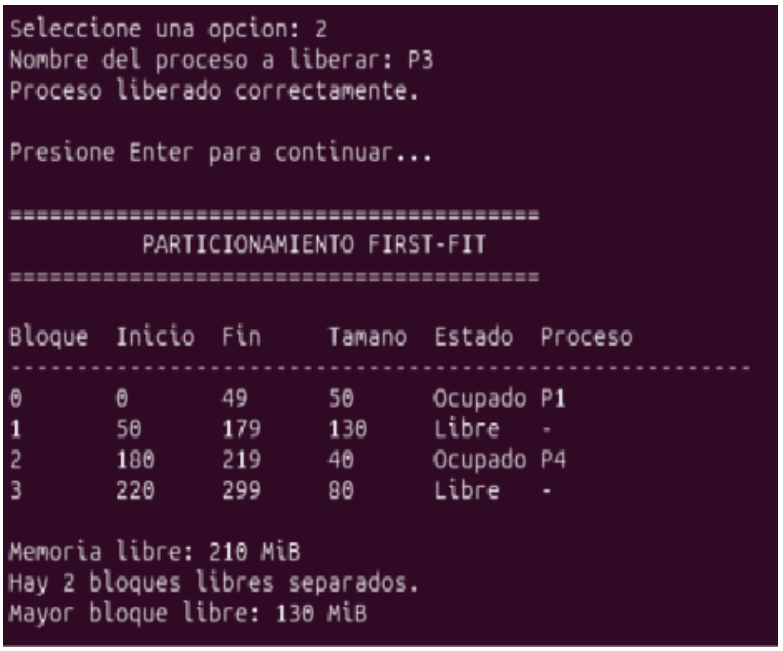
Nota. Al liberar P2 se obtuvieron dos bloques libres separados.

Figura 4.4. Detección de fragmentación externa



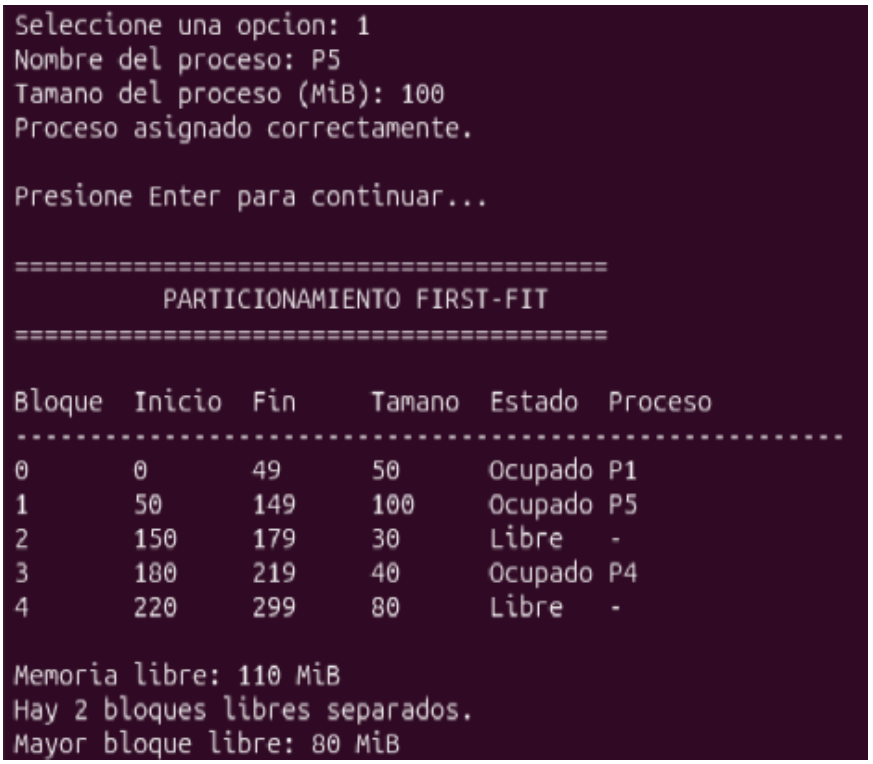
Nota. La solicitud de P5 fue rechazada porque no había un bloque libre contiguo suficiente.

Figura 4.5. Unión de bloques libres contiguos



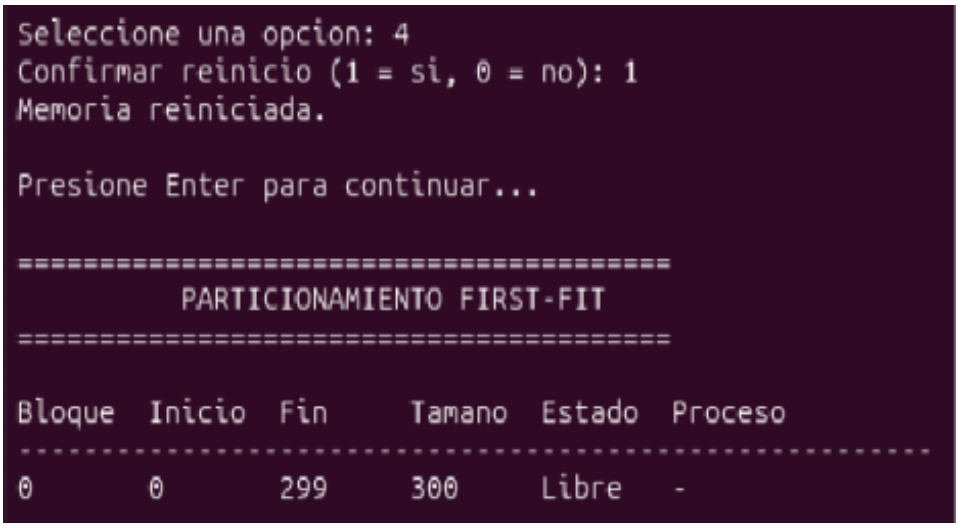
Nota. Después de liberar P3, dos espacios libres contiguos se unieron.

Figura 4.6. Reutilización de memoria mediante First-Fit



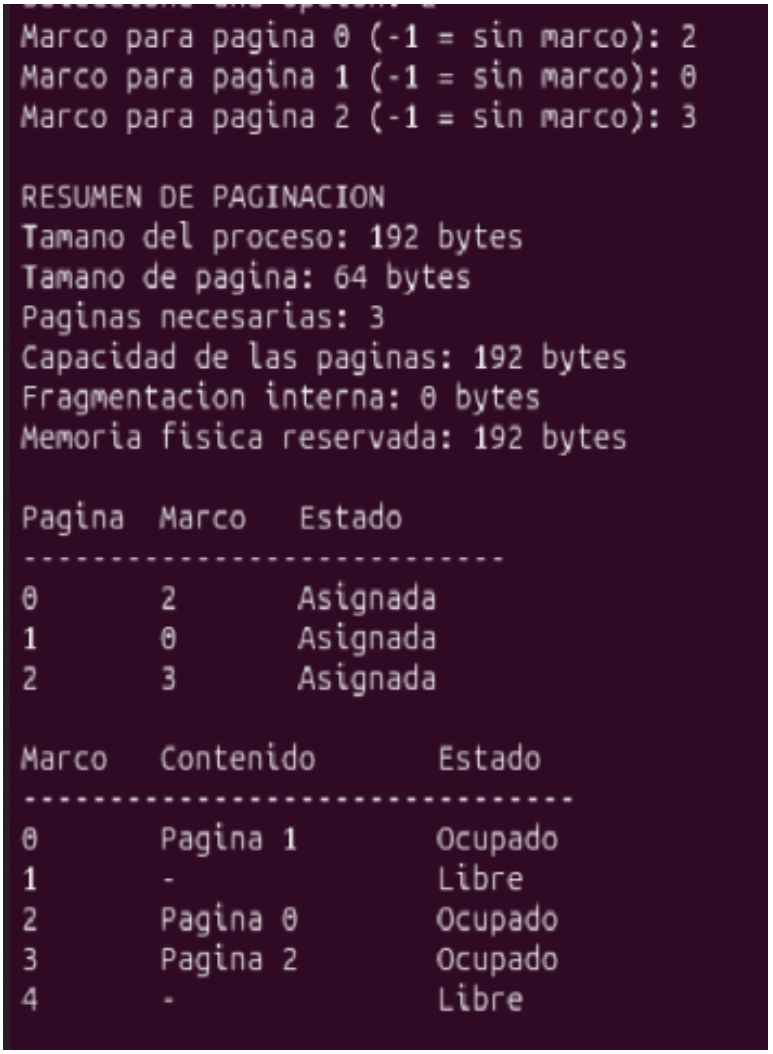
Nota. P5 se asignó al bloque de 130 MiB; quedaron 30 MiB libres en ese espacio.

Figura 4.7. Reinicio de memoria



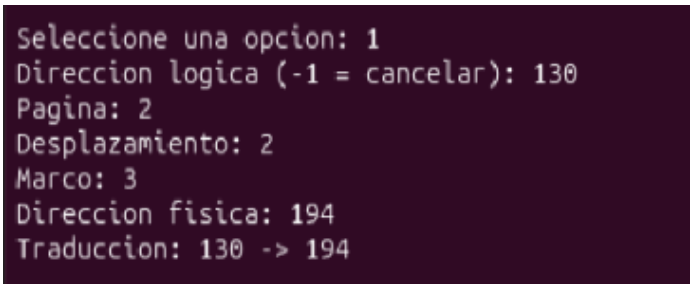
Nota. Al confirmar el reinicio, la memoria volvió a mostrar el total libre.

Figura 4.8. Asignación manual y tabla de páginas



Nota. El proceso de 192 bytes se dividió en tres páginas de 64 bytes, asignadas a los marcos 2, 0 y 3.

Figura 4.9. Traducción de la dirección lógica 130



Nota. La página 2 ubicada en el marco 3, produjo la dirección física 194.